Meeting 10. Data as Arrays

Act 1 A Table of Data Is a Matrix · Act 2 An Image Is an Array · Act 3 A Word Is a Vector

This unit is about computing with data: how data is stored, what an operation on it costs, and which processor carries it out. We begin by storing three kinds of data as **arrays** of numbers: a table, an image, and text. In each case, we represent one observation as an array and identify its shape. We then collect many observations in an array with one additional dimension. The timing experiments establish a useful distinction: the same arithmetic, written in two ways, can differ in speed by a factor of a hundred.

The examples use group 10's dataset, image, and word, as shown in class. The walkthrough follows the three lab notebooks, with each step's code and recorded output available below its explanation. Open a code block when you want to examine the computation.

The notebooks of this meeting.

Each opens in Google Colab. Choose *Save a copy in Drive* in the *File* menu before you change anything, so that your work is kept.

- Act 1: A Table of Data Is a Matrix Steps 1 to 10
- Act 2: An Image Is an Array Steps 11 to 18
- Act 3: A Word Is a Vector Steps 19 to 25
- The class notebook the three short blocks run in class, one for each act

The slides of the three acts are on the meeting's page on Canvas, beside these links. The full notebooks explain every step and test your code with checks.

The main points

The eight main points below link to explanations and code in the walkthrough.

1 Data of every kind becomes an array, and the shape of the array says what the data is

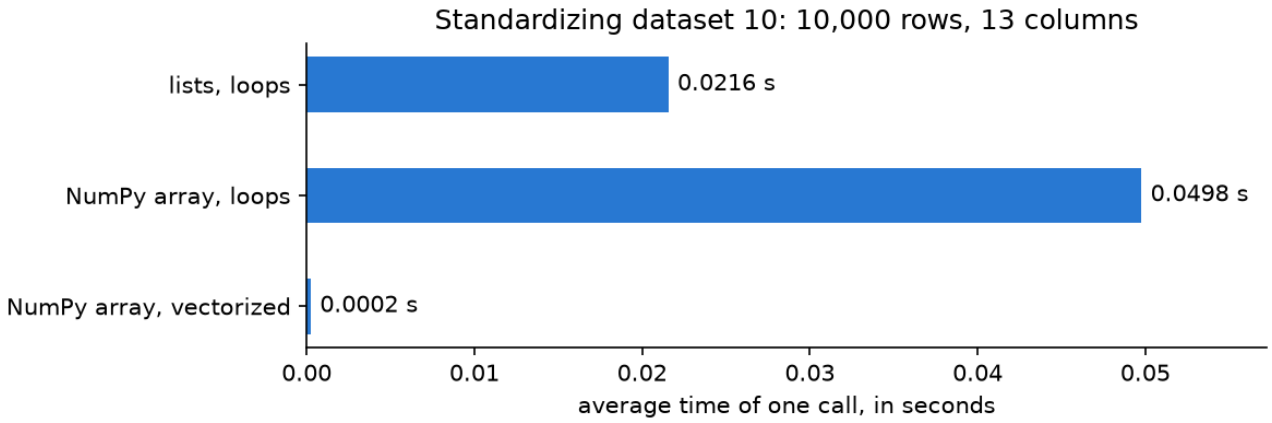
A table with n records and m measurements is a **matrix** with n rows and m columns. A grey image with H rows and W columns of pixels is a matrix of shape $H \times W$. A colour image holds three numbers for each pixel and is an array of shape $H \times W \times 3$. A text of L words, each word replaced by a vector of m numbers, is a matrix of shape $L \times m$. An array with any number of dimensions is called a **tensor**; a matrix is a tensor with two dimensions. In NumPy the shape is one attribute, `X.shape`, and the first question to ask of any array is what each of its dimensions counts.

Data	One point	A batch of N points
a row of a table	m numbers	$N \times m$
a grey image	$H \times W$	$N \times H \times W$
a colour image	$H \times W \times 3$	$N \times H \times W \times 3$
a text of L words	$L \times m$	$N \times L \times m$

Sections 1.2, 2.2, 3.6.

2 Vectorization: write the loop as one operation on the whole array

Standardizing a table means three passes over all of its numbers. Written with `for` loops in Python, the three passes took 166 nanoseconds for each number of dataset 10. Written as one line on the whole array, `(X - X.mean(axis=0)) / X.std(axis=0)`, the same arithmetic took 1.9 nanoseconds for each number: 88 times faster. A computation written as operations on whole arrays, with no Python loop over the entries, is **vectorized**. The loop still exists; it runs inside NumPy. In computations on data, and in everything that is called machine learning, this is the difference between a program that is usable and one that is not.



The three versions of standardization, timed on dataset 10 (10,000 rows, 13 columns).

Sections 1.6 and 1.8.

3 Why the times differ: an interpreter against compiled code

Python does not turn a program into instructions for the processor. It translates each line into small steps called **bytecode**, and a program called the **interpreter** carries them out one at a time. For every pass of a loop, the interpreter reads several steps, looks up what each name refers to, inspects the type of each value to decide what `-` and `/` mean, and creates a new object for each result: 24 bytes for one decimal number. A NumPy array is one block of 8-byte numbers of one type, and an operation on it is a single call into code written in C and compiled in advance, whose loop does none of that work for each number. A Python loop that reads a NumPy array one entry at a time is the slowest of all: it keeps the interpreter's work and adds the building of a Python object for every number it reads. **An array is fast when the loop over its numbers runs inside NumPy.**

Section 1.9.

4 Broadcasting: a shape that mathematics would refuse

In the standardization line, `X - mu` subtracts an array of m numbers from an array with n rows and m columns. In mathematics the shapes do not fit, and the m means would first have to be written again on every row. NumPy does that for you, without storing the n copies: it subtracts `mu[j]` from every entry of column j. This rule for combining arrays of different shapes is **broadcasting**. The simplest case is one number against a whole array: `v + 10` adds 10 to every entry.

OUTPUT

```
shape of X: (10000, 13)   shape of mu: (13,)   shape of X - mu: (10000, 13)
v + 10 = [11. 12. 13. 14.]   one number, added to every entry
```

Section 1.6.

5 Slicing: a row, a column, a channel, a block

`X[0]` is the first row of a matrix, one point. `X[:, 0]` is its first column, one feature. A colon alone means all the positions along that dimension, and `a:b` means the positions a, $a+1$, and so on up to $b-1$. In a colour image, `image[:, :, 0]` is the red channel, a matrix with H rows and W columns, and `image[100:200, 300:400]` is a block of 100 by 100 pixels with all of their channels. A slice is a view of the same numbers, not a copy.

OUTPUT

```
X[0]                    a row, one point:          shape (13,)
X[:, 0]                 a column, one feature:     shape (10000,)
image                   H rows, W columns, 3 channels: (640, 538, 3)
image[:, :, 0]          the red channel:           shape (640, 538)
image[100:200, 300:400] a block of pixels:         shape (100, 100, 3)
```

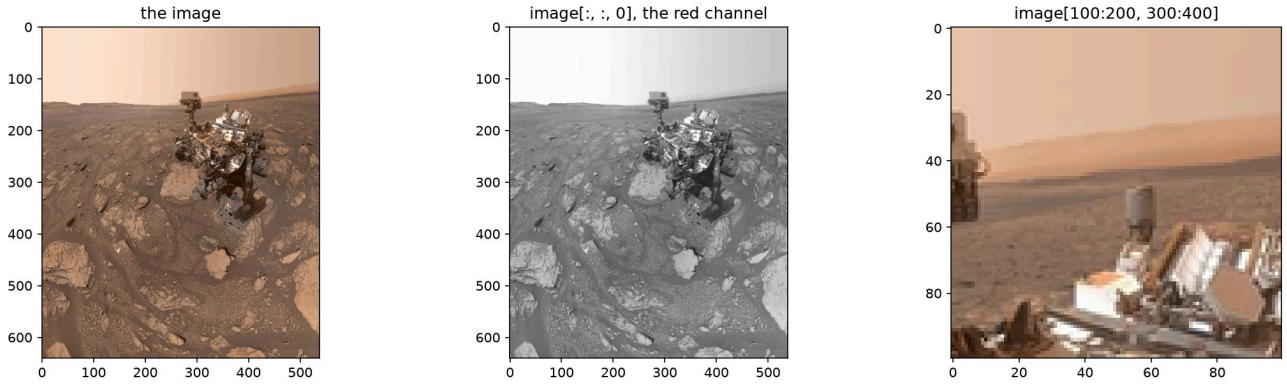


Image 10, its red channel, and the block image[100:200, 300:400].

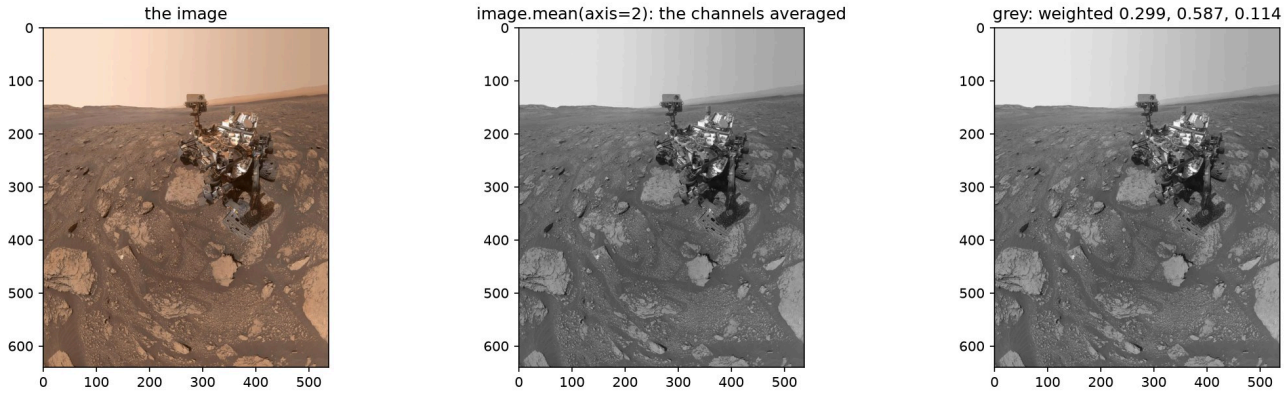
Sections 2.1 and 2.2.

6 Along an axis: the dimension that disappears

`X.mean(axis=0)` runs down the rows and leaves one number for each column: an array of m numbers. `X.mean(axis=1)` runs along the columns and leaves one number for each row. The axis that is named is the dimension that disappears; the others keep their places. On a colour image, `image.mean(axis=2)` averages the three channels of each pixel and leaves a grey picture of shape $H \times W$: not quite the grey picture of the weighted rule of Act 2, but close.

OUTPUT

```
X.mean(axis=0)        down the rows: one number for each column   shape (13,)
X.mean(axis=1)        along the columns: one number for each row  shape (10000,)
image.mean(axis=2)    over the three channels of each pixel        shape (640, 538)
```

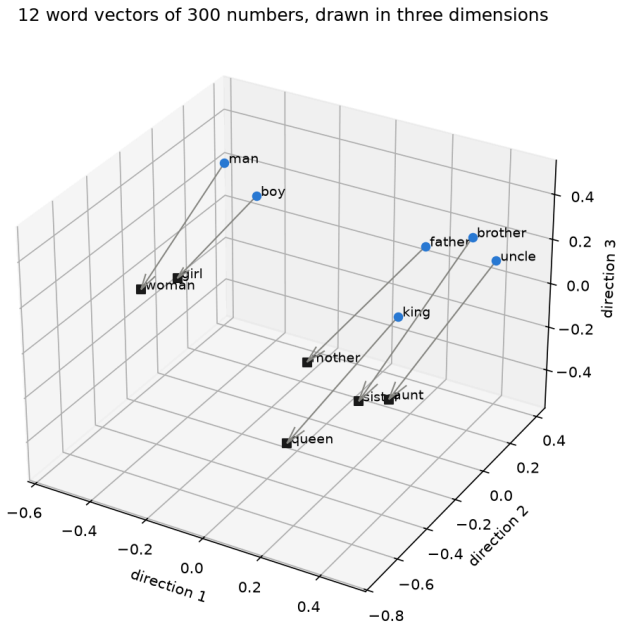


The channels averaged, against the weighted grey picture.

Sections 1.6 and 2.3.

7 Even a word is a vector, and the arithmetic of vectors says something about language

A **word embedding** gives every word of a vocabulary a vector of m numbers; the course uses vectors of 50 and of 300 numbers for 50,000 English words. No one chose the numbers by hand. A program read six billion words of text and gave similar vectors to words that occur in similar surroundings. The words closest to *teacher* are *student*, *graduate*, *teaching* and *taught*, and the vector of *king* minus the vector of *man* plus the vector of *woman* lands nearest to *queen*. One line, `U @ v`, compares one word with all 50,000 at once, by 50,000 dot products.



Twelve word vectors of 300 numbers, drawn in the three directions along which they are most spread out. The arrows are nearly parallel.

Sections 3.2 and 3.3.

8 One point, then a batch: the number of points goes in front

Several points stored and processed together are a **batch**. A batch adds one dimension at the front of the point's own shape, and that dimension counts the points. Every operation of this meeting ran on a whole batch at once: `X - mu` on all the rows of a table, the grey rule on thirty images in one line, `U @ v` on 50,000 words. The table below was printed by the class notebook: a day of weather at Newark airport is two numbers, and 6,940 days are a 6,940 by 2 matrix; a small colour image is 96 by 96 by 3, and thirty of them are 30 by 96 by 96 by 3.

OUTPUT

data	one point	N points	N
a day of weather	(2,)	(6940, 2)	6,940
a colour image	(96, 96, 3)	(30, 96, 96, 3)	30
a word	(300,)	(50000, 300)	50,000
a text of L words	(9, 300)	(3, 9, 300)	3

Sections 2.6 and 3.7.

Working with the notebooks, and with an AI assistant.

Each full notebook is run from the top, one cell at a time, with Shift+Enter. A paragraph that begins with **Your turn** asks for something before you go on, and a cell under the word **Check** tests what you wrote and says what is wrong if something is. In Steps 5, 13 and 15 the notebooks ask an AI assistant to write code. You may use any assistant; the one built into Colab sees the notebook, and so needs less context in the prompt than a separate chat does. A precise prompt is part of the work: it states the name of the function, its arguments, what it may use and what it may not, and what it returns. The notebooks keep the prompt you used in a cell marked **Our prompt**, so that the notebook records how its code was made.

Act 1. A Table of Data Is a Matrix Open the notebook of Act 1

1.1 Thirty datasets, one for each group Steps 1 and 2

The course keeps thirty tabular datasets, numbered 1 to 30, all from the UCI Machine Learning Repository. Each group drew a number at random, and the number names the group's dataset in this act, its image in Act 2 and its word in Act 3. The number of these notes is 10: *Electrical Grid Stability Simulated Data*, in which one row is one simulated state of a small electrical grid with four nodes.

A dataset is a **CSV file**: a text file with one record on each line and commas between the values, with the names of the columns on the first line. **pandas** is the Python library for tables. `pd.read_csv` reads the file into a **DataFrame**, a table held in memory in which every column has a name and holds values of one type; `df.head()` shows its first five rows. pandas does much more than this, with rows and columns to choose, values that are missing, and dates. The unit does not go further into it, for a reason the meeting gave openly: this is the kind of code an AI assistant now writes well from a plain request, and what one must know is what a DataFrame is and what the result should look like. The files of this unit have been prepared so that every column is numeric and no value is missing.

Code: Steps 1 and 2, the dataset and its first rows

cells `dataset-number` , `read`

```
DATASET = 10      # replace 0 with the number of your group's dataset, from 1 to 30

assert 1 <= DATASET <= 30, "Type the number of your group's dataset in place of the 0."

datasets = pd.read_csv(BASE + "datasets.csv", index_col="number")
info = datasets.loc[DATASET]
print(f"Dataset {DATASET}: {info['title']}")
print(f"One row is {info['one_row_is']}.")
print(f"Source: {info['source']}")
```

OUTPUT

```
Dataset 10: Electrical Grid Stability Simulated Data
One row is one simulated state of a small electrical grid with four nodes: reaction times, power, price
coefficients, and a stability measure.
Source: https://doi.org/10.24432/C5PG66
```

```
df = pd.read_csv(BASE + info["file"])
print("rows and columns:", df.shape)
df.head()
```

OUTPUT

```
rows and columns: (10000, 13)
```

	tau1	tau2	tau3	tau4	p1	p2	p3	p4	g1	
0	2.959060	3.079885	8.381025	9.780754	3.763085	-0.782604	-1.257395	-1.723086	0.650456	0.859
1	9.304097	4.902524	3.047541	1.369357	5.067812	-1.940058	-1.872742	-1.255012	0.413441	0.862
2	8.971707	8.848428	3.046479	1.214518	3.405158	-1.207456	-1.277210	-0.920492	0.163041	0.766
3	0.716415	7.669600	4.486641	2.340563	3.963791	-1.027473	-1.938944	-0.997374	0.446209	0.976
4	3.134112	7.608772	4.943759	9.857573	3.525811	-1.125531	-1.845975	-0.554305	0.797110	0.455

1.2 From a table to a matrix Step 3

A **matrix** is a rectangular arrangement of numbers in rows and columns. The table is a matrix with n rows and m columns, and three words are used for each part of it. A **row** is one record; its m numbers are the coordinates of one **point** in a space with m dimensions. A **column** is one measured quantity, also called a **feature** or a **dimension**. The entry in row i and column j is written x_{ij}, and Python counts from 0, so that the first row is row 0 and the first column is column 0. Dataset 10 has $n = 10,000$ points with $m = 13$ coordinates each: 130,000 numbers.

Python stores these numbers in two ways. A **list of lists** is what the language provides on its own: a list of n rows, each a list of m numbers, with the entry `rows[i][j]`. A list can hold anything, letters, strings, other lists, and the numbers in it are Python objects. A **NumPy array** holds numbers only, all of one type, in one block of memory, with the entry `X[i, j]`. The difference looks small on this step, a comma in place of a pair of brackets. It is the difference that the rest of the act measures. NumPy is not only another way to store the block: it is a small language on top of Python, with operations that act on a whole array at once.

Code: Step 3, the same numbers as a list of lists and as an array cell `matrix`

```
X = df.to_numpy(dtype=float)      # a NumPy array of decimal numbers
n, m = X.shape
rows = X.tolist()                 # the same numbers as a list of n lists
ORIGINAL = X.copy()               # a spare copy, used only by the checks

print(f"n = {n:,} rows (points), m = {m} columns (dimensions), n x m = {n * m:,} numbers")
print("type of rows:", type(rows).__name__, "of", type(rows[0]).__name__)
print("type of X:   ", type(X).__name__, "with entries of type", X.dtype)
print("first row, from the list: ", rows[0][:4])
print("first row, from the array:", X[0, :4])
```

OUTPUT

```
n = 10,000 rows (points), m = 13 columns (dimensions), n x m = 130,000 numbers
type of rows: list of list
type of X:    ndarray with entries of type float64
first row, from the list:  [2.95906002455997, 3.07988520422811, 8.38102539191882, 9.78075443222607]
first row, from the array: [2.95906002 3.0798852  8.38102539 9.78075443]
```

1.3 Standardization, and the work it takes Step 4

The columns of a dataset are measured on different scales. **Standardization** brings every column to the same scale, so that columns can be compared and combined. For column j, its mean is the sum of the column divided by n; its standard deviation is the square root of the average squared distance from that mean; and every entry is replaced by its distance from the mean of its column, measured in standard deviations:

$$\mu_j = \frac{1}{n}\sum_{i=1}^{n} x_{ij}, \qquad \sigma_j = \sqrt{\frac{1}{n}\sum_{i=1}^{n} (x_{ij} - \mu_j)^2}, \qquad z_{ij} = \frac{x_{ij} - \mu_j}{\sigma_j}.$$

After standardization every column has mean 0 and standard deviation 1. The standardized values are also called z-scores, which is how a student named them when asked. The word standardization is used for other things too, such as scaling a set of numbers so that they add to 1, like probabilities; in this course it means the three formulas above. The sum for the standard deviation is divided by n, which is what NumPy's `std` does. pandas divides by $n - 1$; both conventions exist, and the course uses n throughout, so a comparison with `df.std()` shows a small difference.

Before any code, the step asks for a count. A program that follows the three formulas with loops passes over the whole matrix three times: once for the means, once for the standard deviations, once to compute the new entries. That is three visits to every number, $3nm$ in all, or four if the writing of each new entry is counted as a visit, as it was in class. For dataset 10 that is $3 \times 130{,}000 = 390{,}000$ visits. At a guessed 100 nanoseconds a visit, one standardization would take about 0.04 seconds. Keep that guess; section 1.8 measures it.

1.4 Version 1: loops on Python lists, written by an AI assistant Step 5

The first version works on the list of lists and uses nothing but the language itself: `for` loops and arithmetic on single numbers, no NumPy, no pandas, no import other than `math`. It is the way the computation would be written in Java or in C, and it is written on purpose at that level, so that every visit to every number is in plain view. The notebook asks an AI assistant to write it, from a prompt that must state four requirements: the name `standardize_loops` and its one argument, a list of n lists of m numbers; loops and single-number arithmetic only; the mean and the standard deviation of each column, dividing by n; and a new list of lists as the result, with the input left as it was.

A request for standardization code should specify the mean and standard deviation of each column, the input format, and the allowed libraries. For this version, request explicit loops over a list of lists, with no NumPy or pandas. State the definition rather than relying on an assistant to infer it from context.

Read the function before running it. Its innermost lines run once for every number of the matrix, in each of the three passes. The check after it tests three things: that the result is a list of lists, that every column of the result has mean 0 and standard deviation 1, and that the input was not changed. A function that divides by $n - 1$ fails the check with a message that says so.

Code: Step 5, the function written to those requirements, and its check cells `paste-standardize-loops`, `run-check-loops`

```
import math


def standardize_loops(rows):
    n = len(rows)
    m = len(rows[0])
    means = [0.0] * m
    for j in range(m):
        total = 0.0
        for i in range(n):
            total += rows[i][j]
        means[j] = total / n
    stds = [0.0] * m
    for j in range(m):
        total = 0.0
        for i in range(n):
            d = rows[i][j] - means[j]
            total += d * d
        stds[j] = math.sqrt(total / n)
    result = []
    for i in range(n):
        new_row = []
        for j in range(m):
            new_row.append((rows[i][j] - means[j]) / stds[j])
        result.append(new_row)
    return result
```

```
assert "standardize_loops" in globals(), "Paste the function standardize_loops into the cell above and run that cell."
assert not mentions_numpy(standardize_loops), (
    "This version must use plain Python lists and loops: no NumPy and no pandas.")

Z_lists = standardize_loops(rows)
assert isinstance(Z_lists, list) and isinstance(Z_lists[0], list), "The function must return a list of lists."
check_standardized(Z_lists, rows)
```

OUTPUT

```
Correct: every column of the result has mean 0 and standard deviation 1.
```

1.5 Version 2: the same loops on a NumPy array Step 6

The second version does exactly what the first does, with the same three passes of loops, on the array `X` instead of the list of lists. It differs in two places: it reads an entry as `X[i, j]` in place of `rows[i][j]`, and it writes its results into a new array, created with `np.empty((n, m))`. The step asks for a prediction before anything is timed: will these loops run faster or slower on the array than on the lists? The tempting answer is faster, because NumPy is the library for fast arithmetic. Write the prediction down; section 1.8 has the measurement and section 1.9 the reason.

Code: Step 6, the loops on the array cell `array-loops`

```
def standardize_array_loops(X):
    n, m = X.shape
    means = [0.0] * m
    for j in range(m):                  # first pass: the mean of each column
        total = 0.0
        for i in range(n):
            total += X[i, j]
        means[j] = total / n
    sds = [0.0] * m
    for j in range(m):                  # second pass: the standard deviation of each column
        total = 0.0
        for i in range(n):
            d = X[i, j] - means[j]
            total += d * d
        sds[j] = math.sqrt(total / n)
    Z = np.empty((n, m))                # a new array for the results
    for i in range(n):                  # third pass: the new entries
        for j in range(m):
            Z[i, j] = (X[i, j] - means[j]) / sds[j]
    return Z


Z_array_loops = standardize_array_loops(X)
check_standardized(Z_array_loops, X)
```

OUTPUT

```
Correct: every column of the result has mean 0 and standard deviation 1.
```

1.6 Version 3: one operation on the whole array Step 7

NumPy applies an operation to every entry of an array in a single call. On a short array `v`, `v + 10` adds 10 to every entry, `v * v` multiplies each entry by itself, and `v.sum()` and `v.mean()` reduce the whole array to one number. Writing a computation this way, as operations on whole arrays with no Python `for` loop over the entries, is called **vectorization**. Standardization needs three such operations:

- `X.mean(axis=0)` computes the mean of every column and returns an array of m numbers. `axis=0` says that the operation runs down the rows, so that one number is left for each column; it is the dimension that disappears.
- `X.std(axis=0)` computes the standard deviation of every column in the same way, dividing by n.
- `X - mu` subtracts an array of m numbers from an array with n rows and m columns. The shapes do not match, and in mathematics the m means would have to be written again on every row before the subtraction. NumPy subtracts `mu[j]` from every entry of column j, as if that had been done, without storing n copies of the means. This rule for combining arrays of different shapes is **broadcasting**, and the division by `sigma` works the same way.

The whole of Versions 1 and 2, three passes of nested loops, is what the one line `(X - mu) / sigma` replaces. The loops have not disappeared. They run inside NumPy, where the next sections measure them.

Other languages.

MATLAB offers the same two ways of writing a computation, with loops or on whole arrays, and the same gap in speed between them; so does R. Vectorization is a habit worth forming in any of these languages, not a feature of NumPy.

Code: Step 7, operations on whole arrays, and the vectorized standardization

cells `vector-operations`, `vectorized`

```
v = np.array([1.0, 2.0, 3.0, 4.0])
print("v      =", v)
print("v + 10 =", v + 10)        # 10 is added to every entry
print("v * v  =", v * v)         # each entry is multiplied by itself
print("sum    =", v.sum())
print("mean   =", v.mean())
```

OUTPUT

```
v      = [1. 2. 3. 4.]
v + 10 = [11. 12. 13. 14.]
v * v  = [ 1.  4.  9. 16.]
sum    = 10.0
mean   = 2.5
```

```
def standardize_vectorized(X):
    mu = X.mean(axis=0)          # the m column means
    sigma = X.std(axis=0)        # the m column standard deviations
    return (X - mu) / sigma      # broadcasting: n rows and m columns, combined with m numbers


mu = X.mean(axis=0)
print("shape of X:     ", X.shape)
print("shape of mu:    ", mu.shape)
print("shape of X - mu:", (X - mu).shape)

Z = standardize_vectorized(X)
check_standardized(Z, X)
```

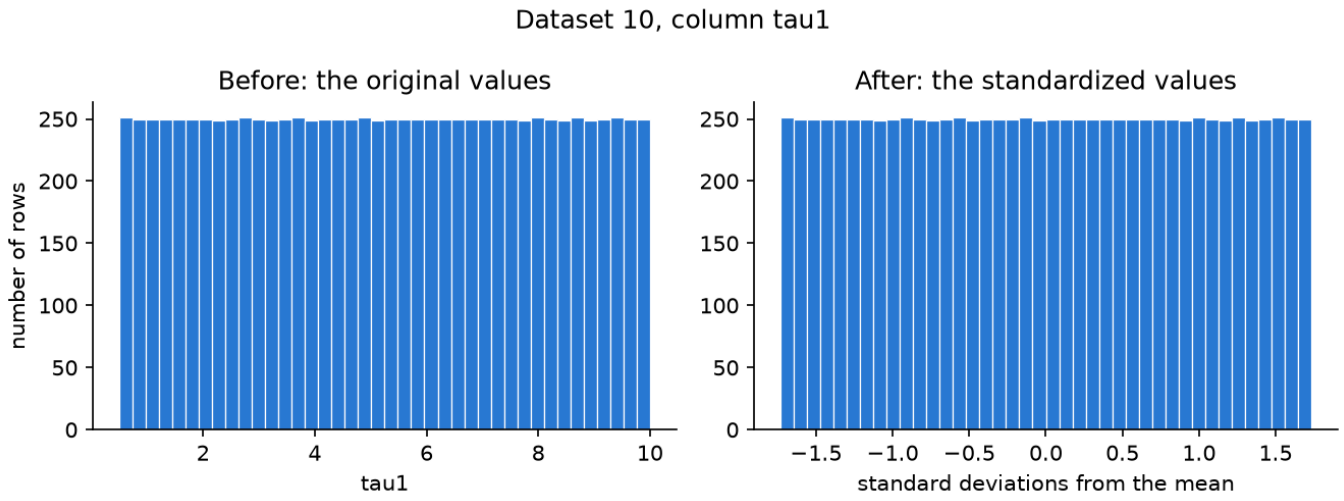
OUTPUT

```
shape of X:      (10000, 13)
shape of mu:     (13,)
shape of X - mu: (10000, 13)
Correct: every column of the result has mean 0 and standard deviation 1.
```

1.7 What standardization does to one column Step 8

A **histogram** divides the range of a column's values into intervals of equal width and draws, for each interval, a bar whose height is the number of rows with a value in that interval. The two histograms below are the first column of dataset 10 before and after standardization. They have the same shape. Standardization shifts the

values and changes their scale; it does not change their order or how they are spread relative to each other. Only the numbers on the horizontal axis change: from the original values to standard deviations from the mean.



Column 0 of dataset 10, before and after standardization.

In the printed table, the mean after standardization appears as a number such as `-1.8e-16` rather than exactly 0. The difference is the rounding error of arithmetic on decimal numbers in a computer, and it is the size to expect: about 10^{-16} relative to the numbers involved.

Code: Step 8, one column before and after cell `histograms`

```
COLUMN = 0      # the position of a column, from 0 to m - 1

name = df.columns[COLUMN]
before = X[:, COLUMN]            # all rows, one column
after = Z[:, COLUMN]

print(f"Dataset {DATASET}, column {COLUMN}: {name}")
print(f"{'':8s}{'minimum':>14s}{'mean':>14s}{'maximum':>14s}{'std. deviation':>16s}")
for label, values in (("before", before), ("after", after)):
    print(f"{label:8s}{values.min():14.4g}{values.mean():14.4g}{values.max():14.4g}{values.std():16.4g}")

fig, (left, right) = plt.subplots(1, 2, figsize=(9, 3.4))
left.hist(before, bins=40, color="#2a78d6", edgecolor="white", linewidth=0.6)
left.set_title("Before: the original values")
left.set_xlabel(name)
left.set_ylabel("number of rows")
right.hist(after, bins=40, color="#2a78d6", edgecolor="white", linewidth=0.6)
right.set_title("After: the standardized values")
right.set_xlabel("standard deviations from the mean")
for ax in (left, right):
    ax.spines[["top", "right"]].set_visible(False)
fig.suptitle(f"Dataset {DATASET}, column {name}")
fig.tight_layout()
plt.show()
```

OUTPUT

```
Dataset 10, column 0: tau1
                minimum          mean       maximum  std. deviation
before           0.5008          5.25         9.999           2.742
after            -1.732    -1.787e-16         1.732               1
```

1.8 Measuring the three versions Step 9

One measurement of a short computation is not reliable. The time changes from run to run, because the machine does other work at the same moment, and a computation of milliseconds is easily disturbed. The remedy is to repeat the computation several times and report the average. The function `average_time` does that with `time.perf_counter()`, which returns the reading of a precise clock in seconds; the difference between two readings is the time that passed between them.

The table gives, for each version, the time of one call and the **time per number**: the time of one call divided by $n \times m$, in nanoseconds, a nanosecond being one billionth of a second. The time of one call depends on the size of the dataset; the time per number changes little with it, so groups with different datasets can compare their results. On the machine that built these notes, for dataset 10:

- the loops on lists took 166 nanoseconds for each number, for all three visits together: less than the 300 that the guess of 100 nanoseconds a visit predicted;
- the same loops on the NumPy array took 383 nanoseconds for each number. The loops on the array are the **slowest** of the three, on this dataset and on all thirty;
- the vectorized line took 1.9 nanoseconds for each number: 88 times faster than the lists and 202 times faster than the loops on the array.

Your own numbers differ, with your machine and your dataset; the ratios are what to compare. On Colab's machines the loops are slower still and the ratios larger.

Counted as additions, subtractions, multiplications and divisions on the matrix entries, all three versions perform the same arithmetic: the same $3nm$ visits. Python loops also incur interpreter overhead. Vectorization changes the time for that arithmetic, not the arithmetic itself.

Code: Step 9, the timer, the measurement, and the chart cells `average-time`, `measure`, `timing-chart`

```python
def average_time(function, data, repeats=5):
    """The average time of one call function(data), in seconds, over several calls."""
    total = 0.0
    for _ in range(repeats):
        start = time.perf_counter()              # the clock before the call
        function(data)
        total += time.perf_counter() - start     # the time that this call took
    return total / repeats


waited = average_time(time.sleep, 0.05, 3)       # a test: time.sleep(0.05) only waits for 0.05 seconds
print(f"A wait of 0.05 seconds was measured as {waited:.4f} seconds.")
```

OUTPUT

```
A wait of 0.05 seconds was measured as 0.0502 seconds.
```

```
t_lists = average_time(standardize_loops, rows, 5)
t_array_loops = average_time(standardize_array_loops, X, 5)
t_vectorized = average_time(standardize_vectorized, X, 20)

times = {
    "lists, loops": t_lists,
    "NumPy array, loops": t_array_loops,
    "NumPy array, vectorized": t_vectorized,
}

print(f"Dataset {DATASET}: n = {n:,} rows, m = {m} columns, n x m = {n * m:,} numbers")
print(f"{'version':26s}{'seconds per call':>18s}{'nanoseconds per number':>25s}{'ratio to vectorized':>22s}")
for version, t in times.items():
    print(f"{version:26s}{t:18.4f}{t / (n * m) * 1e9:25.1f}{t / t_vectorized:22.1f}")
```

OUTPUT

```
Dataset 10: n = 10,000 rows, m = 13 columns, n x m = 130,000 numbers
version                    seconds per call   nanoseconds per number   ratio to vectorized
lists, loops                         0.0216                    165.8                  87.5
NumPy array, loops                   0.0498                    382.8                 202.0
NumPy array, vectorized              0.0002                      1.9                   1.0
```

```
versions = list(times)
seconds = list(times.values())

fig, ax = plt.subplots(figsize=(8, 2.8))
bars = ax.barh(versions, seconds, height=0.5, color="#2a78d6")
ax.bar_label(bars, labels=[f"{t:.4f} s" for t in seconds], padding=4)
ax.invert_yaxis()                # first version at the top
ax.margins(x=0.15)               # room for the labels at the ends of the bars
ax.set_xlabel("average time of one call, in seconds")
ax.set_title(f"Standardizing dataset {DATASET}: {n:,} rows, {m} columns")
ax.spines[["top", "right"]].set_visible(False)
fig.tight_layout()
plt.show()

print("Line for the discussion board:")
print(f"Dataset {DATASET} ({info['title']}), n = {n:,}, m = {m}. Nanoseconds per number: "
      f"lists {t_lists / (n * m) * 1e9:.1f}, array with loops {t_array_loops / (n * m) * 1e9:.1f}, "
      f"vectorized {t_vectorized / (n * m) * 1e9:.1f}.")
```

OUTPUT

```
Line for the discussion board:
Dataset 10 (Electrical Grid Stability Simulated Data), n = 10,000, m = 13. Nanoseconds per number: lists 165.8,
array with loops 382.8, vectorized 1.9.
```

1.9 Why the times differ Step 10

What Python does for each pass of a loop. Python is an **interpreted** language. It does not turn a program into instructions for the processor, as a compiler for C does. It translates each line into small steps called **bytecode**, and another program, the **interpreter**, carries out those steps one at a time. The notebook prints the bytecode of one line, `z = (x - mu) / sigma`: it is eight steps. For every pass of that line the interpreter does all of the following. It reads each step and decides what to do for it. It finds the objects that the names `x`, `mu` and `sigma` refer to. It inspects the **type** of each object to decide what `-` and `/` mean, because a Python name can refer to a value of any type, and the same line could be subtracting dates or joining strings. And it creates a **new object** for each result: a decimal number in Python is an object of 24 bytes, and a list holds a further 8-byte reference to each of its numbers. In `standardize_loops` all of this is repeated for every number of the matrix, three times.

What NumPy does for one operation. A NumPy array stores its numbers in one block of memory, 8 bytes for each number, all of the same type, one after another: the list of lists of dataset 10 occupies 4.8 megabytes, the array 1.0. An operation such as `X - mu` is one call from Python into NumPy's own code, which was written in the language C and translated in advance into instructions for the processor. That code runs the loop over the block. It does not inspect a type for each number, it does not create an object for each number, and the processor can apply one instruction to several neighbouring numbers at once. For one operation that the processor could do in a step, the interpreter does tens of steps; the compiled loop does the one.

Why the loops on the array are the slowest. In `standardize_array_loops` the loop still runs in the interpreter, with all of the work above. Each time it reads `X[i, j]`, NumPy has to find one number in its block and build a new Python object for it, which takes longer than reading an object that already sits in a list. The array gains nothing from being an array when it is read one number at a time.

An array is fast when the loop over its numbers runs inside NumPy.

This is the one thing to carry out of Act 1. Whatever is computed on data in this unit, and in machine learning generally, is written as operations on whole arrays, because a model that was written with Python loops would be a hundred times too slow to train or to use.

Code: Step 10, the bytecode of one line, and the memory of the two representations

cell `bytecode-and-memory`

```
import dis

print("The bytecode of one line:")
dis.dis("z = (x - mu) / sigma")

one_number = sys.getsizeof(rows[0][0])
list_bytes = sys.getsizeof(rows) + sum(sys.getsizeof(row) for row in rows) + n * m * one_number
print()
print(f"One decimal number as a Python object: {one_number} bytes")
print(f"One number in the array X:             {X.itemsize} bytes")
print(f"All of rows (lists and numbers):       {list_bytes / 1e6:.1f} megabytes")
print(f"All of X:                              {X.nbytes / 1e6:.1f} megabytes")
```

OUTPUT

```
The bytecode of one line:
  0           0 RESUME                   0

  1           2 LOAD_NAME                0 (x)
              4 LOAD_NAME                1 (mu)
              6 BINARY_OP               10 (-)
             10 LOAD_NAME                2 (sigma)
             12 BINARY_OP               11 (/)
             16 STORE_NAME               3 (z)
             18 RETURN_CONST             0 (None)

One decimal number as a Python object: 24 bytes
One number in the array X:             8 bytes
All of rows (lists and numbers):       4.8 megabytes
All of X:                              1.0 megabytes
```

Act 2. An Image Is an Array Open the notebook of Act 2

2.1 A grey image is a matrix Step 11

A digital image is a grid of small squares, the **pixels**. In a **grey image** every pixel holds one number, its brightness: 0 is black, 1 is white, and the numbers between are shades of grey. A grey image with H rows and W columns of pixels is therefore a matrix with H rows and W columns. In Act 1 a row of the matrix was one record; here a row is one horizontal line of pixels, and a picture is drawn from the top row down, so that row 0 is at the top. The image file stores each brightness as a whole number from 0 to 255; dividing by 255 brings it to the range from 0 to 1.

The course keeps thirty images, numbered like the datasets: fifteen paintings, from the Art Institute of Chicago, and fifteen photographs from NASA. Image 10 is the selfie that the Curiosity rover took on Mars in 2020. Its grey version has $H = 640$ rows and $W = 538$ columns: 344,320 numbers, the darkest 0.03 and the brightest 1.00, with mean 0.51. The picture below shows the matrix drawn as an image, and beside it a block of 12 by 12 pixels from the middle, enlarged until single pixels can be seen. The block is `grey[r:r + 12, c:c + 12]`, a slice of the matrix, and its numbers are printed under the picture: the larger the number, the lighter the square.

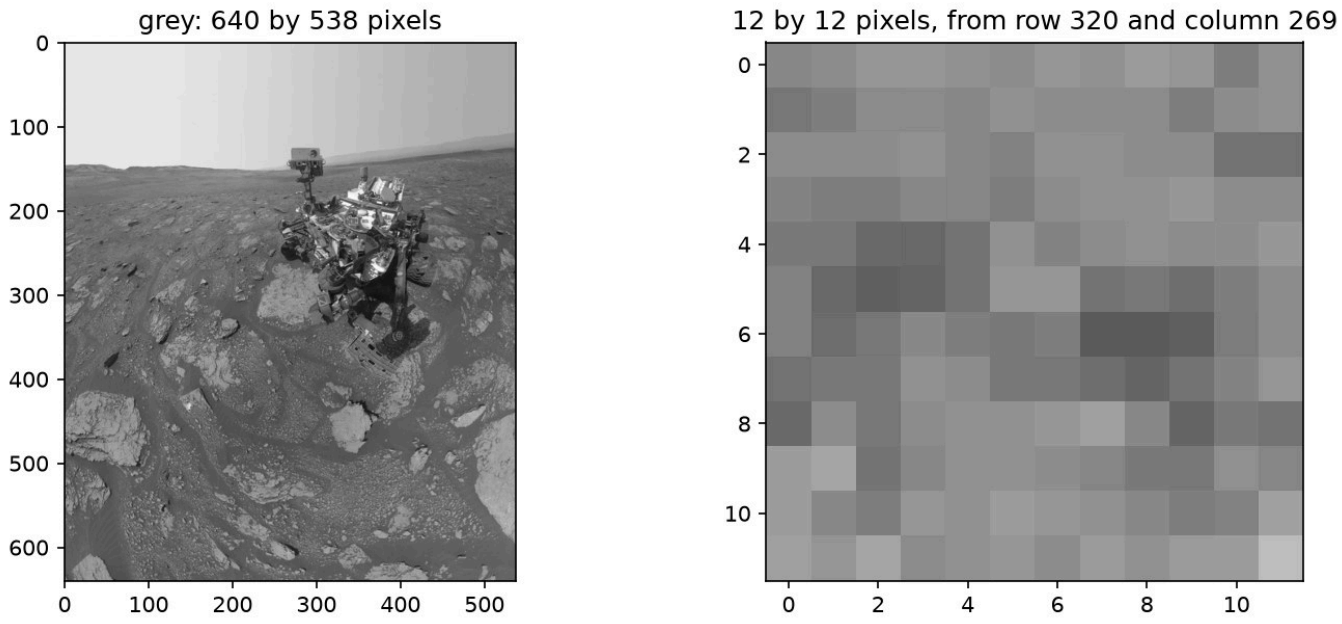


Image 10 in grey, and twelve by twelve of its pixels.

Code: Step 11, the grey image and a block of its pixels

cells `image-number`, `show-grey`

```
IMAGE = 10        # replace 0 with your group's number, from 1 to 30

assert 1 <= IMAGE <= 30, "Type your group's number in place of the 0."

images = pd.read_csv(IMAGES + "images.csv", index_col="number")
picture_info = images.loc[IMAGE]
print(f"Image {IMAGE}: {picture_info['title']} ({picture_info['creator']})")
print(f"Source: {picture_info['source']}")

grey = read_image(picture_info["grey_file"])
H, W = grey.shape
print(f"shape of grey: {grey.shape}, which is H = {H} rows and W = {W} columns of pixels, {H * W:,} numbers")
print(f"darkest value {grey.min():.2f}, brightest value {grey.max():.2f}, mean {grey.mean():.2f}")
```

OUTPUT

```
Image 10: Curiosity's Selfie on Mars (NASA/JPL-Caltech/MSSS)
Source: https://images.nasa.gov/details/PIA24173
shape of grey: (640, 538), which is H = 640 rows and W = 538 columns of pixels, 344,320 numbers
darkest value 0.03, brightest value 1.00, mean 0.51
```

```
r, c = H // 2, W // 2                    # the middle of the image
block = grey[r:r + 12, c:c + 12]         # 12 rows and 12 columns

show([grey, block], [f"grey: {H} by {W} pixels", f"12 by 12 pixels, from row {r} and column {c}"])
print("The numbers of the block, rounded to two decimals:")
print(np.round(block, 2))
```

OUTPUT

```
The numbers of the block, rounded to two decimals:
[[0.55 0.55 0.6  0.59 0.58 0.56 0.59 0.58 0.61 0.59 0.5  0.57]
 [0.47 0.51 0.55 0.55 0.54 0.58 0.56 0.56 0.55 0.5  0.56 0.57]
 [0.55 0.56 0.56 0.58 0.54 0.52 0.58 0.57 0.56 0.56 0.46 0.45]
 [0.53 0.51 0.51 0.54 0.53 0.5  0.55 0.57 0.58 0.6  0.55 0.56]
 [0.49 0.48 0.42 0.41 0.46 0.58 0.53 0.55 0.57 0.55 0.56 0.6 ]
 [0.52 0.42 0.39 0.4  0.47 0.6  0.59 0.46 0.47 0.45 0.51 0.55]
 [0.51 0.43 0.47 0.55 0.51 0.48 0.51 0.36 0.36 0.38 0.49 0.55]
 [0.45 0.48 0.49 0.58 0.56 0.48 0.47 0.44 0.41 0.45 0.53 0.59]
 [0.42 0.55 0.47 0.55 0.57 0.58 0.6  0.64 0.55 0.41 0.49 0.45]
 [0.62 0.65 0.47 0.54 0.58 0.58 0.56 0.53 0.49 0.47 0.57 0.53]
 [0.62 0.55 0.5  0.59 0.57 0.62 0.59 0.58 0.55 0.51 0.51 0.64]
 [0.63 0.59 0.65 0.55 0.58 0.6  0.56 0.61 0.58 0.61 0.62 0.75]]
```

2.2 A colour image is an array with three dimensions Step 12

The eye has three kinds of receptor for colour, and a screen makes every colour from three lights: red, green and blue. In a colour image every pixel therefore holds **three** numbers, how much red, how much green and how much blue it gives, each from 0 to 1. The image is an array with three dimensions, of shape $H \times W \times 3$. `image[i, j]` holds the three numbers of the pixel in row i and column j. `image[:, :, 0]` is the **red channel**, a matrix with H rows and W columns that holds the red number of every pixel; a colon alone means all the positions along that dimension. `image[:, :, 1]` and `image[:, :, 2]` are the green and the blue channel. A colour image is three grey images stacked, one for each colour, and the third dimension is the colour. An array with any number of dimensions is also called a **tensor**; a matrix is a tensor with two dimensions.

Each channel is a matrix, so it can be drawn in grey: a pixel is light in the picture of the red channel where the image holds much red. For the Mars selfie the three channels differ a great deal. The ground is red, so the red channel is the brightest of the three, with mean 0.60 against 0.48 for green and 0.40 for blue, and it shows the texture of the rocks in detail; the green and the blue channels show less of it. The pixel in the top left corner is `[1.00, 0.89, 0.80]`: a pale orange sky.

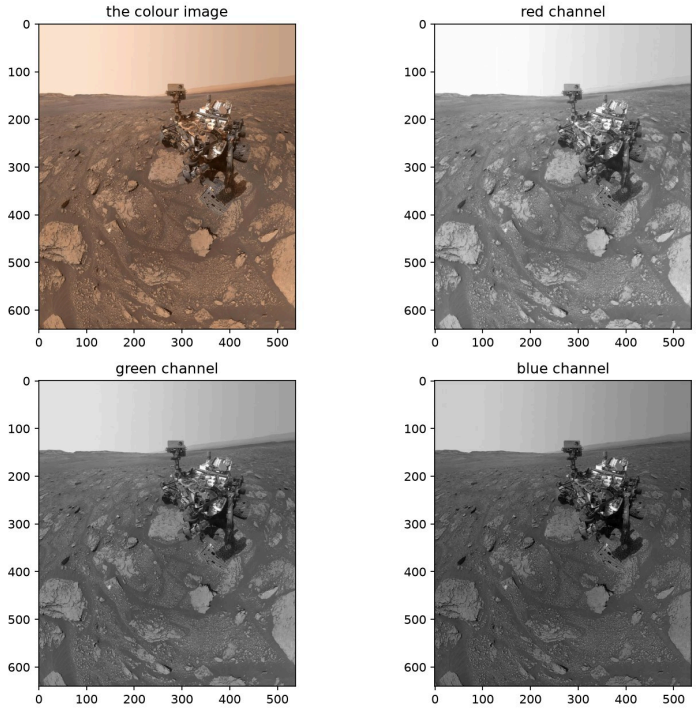


Image 10 and its three channels, each drawn as a grey picture.

Code: Step 12, the colour image and its channels cell `read-colour`

```
image = read_image(picture_info["file"])
print(f"shape of image: {image.shape}, which is {image.size:,} numbers")
print("the pixel in row 0 and column 0:", np.round(image[0, 0], 2), "(red, green, blue)")

red, green, blue = image[:, :, 0], image[:, :, 1], image[:, :, 2]
print("shape of one channel:", red.shape)
print(f"mean of each channel: red {red.mean():.2f}, green {green.mean():.2f}, blue {blue.mean():.2f}")

show([image, red, green, blue], ["the colour image", "red channel", "green channel", "blue channel"])
```

OUTPUT

```
shape of image: (640, 538, 3), which is 1,032,960 numbers
the pixel in row 0 and column 0: [1.   0.89 0.8 ] (red, green, blue)
shape of one channel: (640, 538)
mean of each channel: red 0.60, green 0.48, blue 0.40
```

2.3 From colour to grey, in two ways Step 13

A grey image can be computed from a colour image. A common rule gives every pixel the brightness

$$\text{grey} = 0.299 \cdot \text{red} + 0.587 \cdot \text{green} + 0.114 \cdot \text{blue}\,.$$

The three weights add to 1, and green has the largest because the eye is most sensitive to green light. The grey image of section 2.1 was made from the colour image with this rule. The step asks an AI assistant for two functions in one prompt: `grey_loops`, with two nested loops over the rows and the columns, which reads

the three numbers of each pixel and stores the brightness in a new array made with `np.empty((H, W))`; and `grey_vectorized`, which computes the same result with no loop, by arithmetic on whole channels. The second is one line: the rule itself, with the three channels in place of the three numbers of one pixel.

The check compares both results with the stored grey image, and then times them. On image 10 the loops took 372 nanoseconds for each pixel and the arithmetic on whole channels 10: the loops take 39 times as long. The same gap as in Act 1, for the same reason: the loop over the pixels runs in the interpreter or it runs inside NumPy.

Code: Step 13, the two functions written to the requirements, and their check

cells `paste-grey-functions`, `run-check-grey`

```
def grey_loops(image):
    H, W = image.shape[0], image.shape[1]
    result = np.empty((H, W))
    for i in range(H):
        for j in range(W):
            result[i, j] = 0.299 * image[i, j, 0] + 0.587 * image[i, j, 1] + 0.114 * image[i, j, 2]
    return result


def grey_vectorized(image):
    return 0.299 * image[:, :, 0] + 0.587 * image[:, :, 1] + 0.114 * image[:, :, 2]
```

```
assert "grey_loops" in globals() and "grey_vectorized" in globals(), (
    "Paste the functions grey_loops and grey_vectorized into the cell above and run that cell.")

for function in (grey_loops, grey_vectorized):
    result = np.asarray(function(image), dtype=float)
    assert result.shape == (H, W), f"{function.__name__} returned an array of shape {result.shape}; it must be {(H, W)}."
    worst = np.abs(result - grey).max()
    assert worst < 0.005, (
        f"{function.__name__} differs from the grey image of Step 11 by up to {worst:.3f}. "
        "Check the three weights and the order of the channels.")
print("Correct: both functions reproduce the grey image of Step 11.")

t_grey_loops = average_time(grey_loops, image, 3)
t_grey_vectorized = average_time(grey_vectorized, image, 20)
print()
print(f"Image {IMAGE}: {H} by {W} pixels")
print(f"{'version':14s}{'seconds per call':>18s}{'nanoseconds per pixel':>24s}")
print(f"{'loops':14s}{t_grey_loops:18.4f}{t_grey_loops / (H * W) * 1e9:24.1f}")
print(f"{'vectorized':14s}{t_grey_vectorized:18.4f}{t_grey_vectorized / (H * W) * 1e9:24.1f}")
print(f"The loops take {t_grey_loops / t_grey_vectorized:.0f} times as long as the vectorized version.")
```

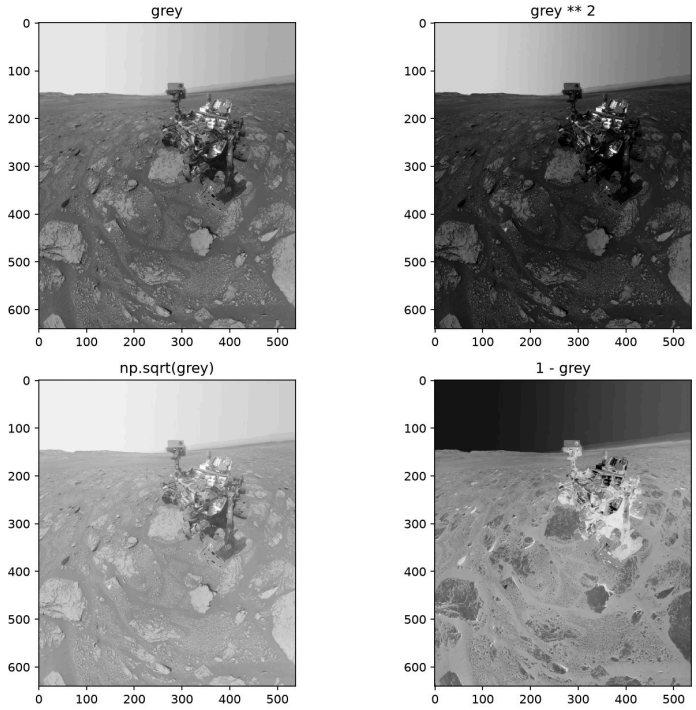
OUTPUT

```
Correct: both functions reproduce the grey image of Step 11.

Image 10: 640 by 538 pixels
version         seconds per call   nanoseconds per pixel
loops                     0.1281                   372.0
vectorized                0.0033                     9.5
The loops take 39 times as long as the vectorized version.
```

2.4 One function applied to every entry Step 14

In Act 1 the standard deviation needed the square of every distance from the mean, and the loops computed `d * d` for one entry at a time. On an array, `grey ** 2` squares every entry in one operation. An operation that applies the same function to each entry separately is **pointwise**. A pointwise operation on a grey image gives a new image of the same shape, and because every brightness lies between 0 and 1, three of them have a visible meaning. `grey ** 2` makes every value smaller, except 0 and 1, which stay as they are: the picture becomes darker, and the mean brightness of image 10 falls from 0.51 to 0.30. `np.sqrt(grey)` makes every value larger, except 0 and 1: the picture becomes lighter, with mean 0.70. `1 - grey` exchanges dark and light: the picture becomes its negative.



The grey image, grey ** 2, np.sqrt(grey) and 1 - grey.

Code: Step 14, three pointwise operations

cell `pointwise`

```
darker = grey ** 2
lighter = np.sqrt(grey)
negative = 1 - grey

print(f"mean brightness: grey {grey.mean():.2f}, grey ** 2 {darker.mean():.2f}, "
      f"np.sqrt(grey) {lighter.mean():.2f}, 1 - grey {negative.mean():.2f}")
show([grey, darker, lighter, negative], ["grey", "grey ** 2", "np.sqrt(grey)", "1 - grey"])
```

OUTPUT

```
mean brightness: grey 0.51, grey ** 2 0.30, np.sqrt(grey) 0.70, 1 - grey 0.49
```

2.5 Your own transformation Step 15

The step asks each group for a new picture computed from the channels of its colour image, by a function `our_transform` that takes the array and returns an array with the same height and width; a value below 0 or above 1 is drawn as 0 or as 1. The notebook suggests starting points: exchange two channels, set one channel to 0, apply a pointwise operation to one channel only, or replace one channel by the grey image. The pictures below are the example of these notes: red and blue exchanged, and the green channel squared. Every such transformation is one or two operations on whole arrays, with the loop over the pixels inside NumPy.

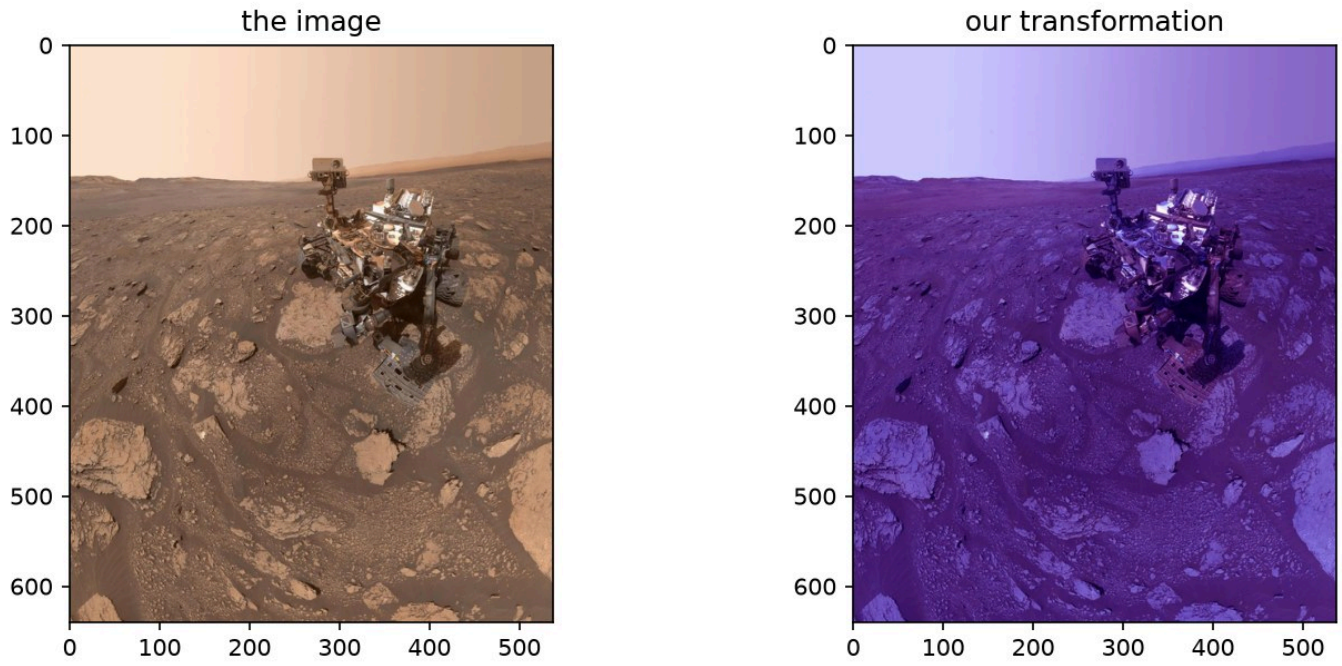


Image 10, and the transformation of these notes: red and blue exchanged, green squared.

A flip changes pixel positions rather than their values: `image[:, ::-1]` reverses the columns, and `image[::-1]` reverses the rows. Copying the red channel into the green channel gives two copies of one channel alongside the blue channel. These transformations require no explicit loop.

To make an image darker toward its centre and fainter toward its edges, each new pixel value must depend on *where* the pixel is. This differs from squaring every pixel by the same rule. Consider how to express the change as a function of position and apply it to the whole array.

Explain what the function does to the array: which channel is read, which is written, and what happens to each pixel.

Code: Step 15, the transformation of these notes cell `our-transform`

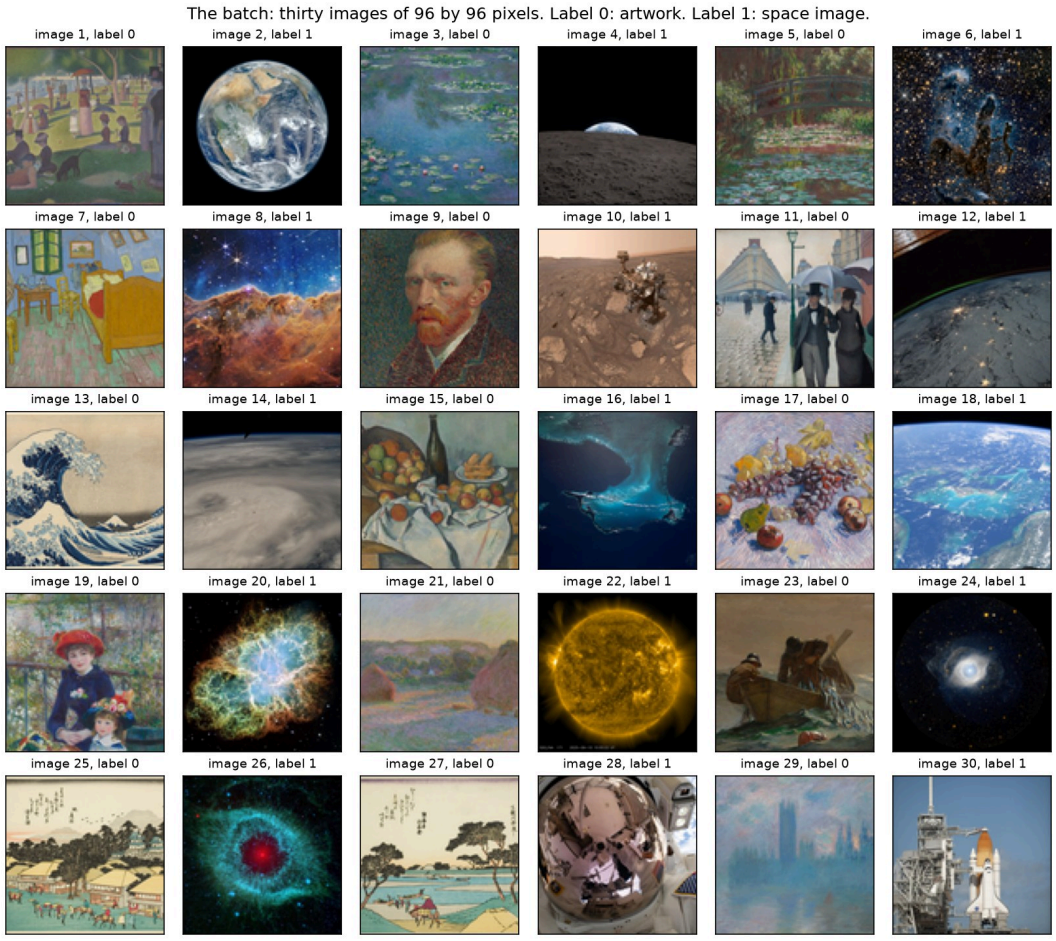
```
def our_transform(image):
    new = image.copy()               # a copy, so that image stays as it was
    new[:, :, 0] = image[:, :, 2]    # red and blue exchanged
    new[:, :, 2] = image[:, :, 0]
    new[:, :, 1] = image[:, :, 1] ** 2
    return new
```

```
new_picture = our_transform(image)
assert new_picture.shape[:2] == (H, W), "The new picture must have the same height and width as the image."
show([image, new_picture], ["the image", "our transformation"])
```

2.6 One image is one point; a batch of images Steps 16 and 17

In Act 1 every row of the matrix was one point, a record with m coordinates. An image is one point too. Its coordinates are all of its numbers, $H \times W \times 3$ of them: 1,032,960 for image 10. `image.reshape(-1)` writes the same numbers as a single row. A point can carry a **label**, a value that says of which kind the point is, and each of the thirty images has one: 0 for an artwork, 1 for an image taken in space. The label of image 10 is 1.

Several points that are stored and processed together are a **batch**. In Act 1 the batch was the whole matrix: n points in n rows. A batch of colour images is an array with **four** dimensions, image, row, column and channel. To fit in one array the images must all have the same size, so the thirty images of the course were each cut to a square and reduced to 96 by 96 pixels. The batch has shape $30 \times 96 \times 96 \times 3$, 829,440 numbers, and `batch[0]`, the first image, has shape $96 \times 96 \times 3$.



The batch: thirty images of 96 by 96 pixels, with their labels.

One more colon, and the grey rule of section 2.3 converts all thirty images at once: `0.299 * batch[:, :, :, 0] + ...` reads the red channel of every image. Then `mean(axis=(1, 2))` averages over the rows and the columns of each image and leaves one number for each: its mean brightness, an array of

shape `(30,)`. The artworks have mean brightness 0.49 and the space images 0.30, and the rule *darker than 0.35 means a space image* gives the right label for 23 of the 30. A rule of this kind, which computes one number from a point and compares it with a threshold, is called a **neuron**; it is the subject of meeting 11. The batch is also the matrix of Act 1: `batch.reshape(30, -1)` writes every image as one row, and the result is 30 points with 27,648 coordinates each.

A transparency channel changes an image from RGB to RGBA, but the array still has three dimensions: row, column, channel. A video adds a fourth dimension for frames; a batch of images adds one for images. The order of frames carries information, while the order of independent images in a dataset usually does not.

Code: Steps 16 and 17, one point, the batch, and one operation for the whole batch

cells `one-point`, `batch`, `batch-operations`

```
point = image.reshape(-1)             # the same numbers, written as one row
print(f"image has shape {image.shape}. As one point, it has {point.shape[0]:,} coordinates.")
print(f"The label of image {IMAGE} is {picture_info['label']}: {picture_info['kind']}.")
```

OUTPUT

```
image has shape (640, 538, 3). As one point, it has 1,032,960 coordinates.
The label of image 10 is 1: space image.
```

```
batch = read_array("batch-96.npy")
labels = images["label"].to_numpy()

print("shape of batch:", batch.shape, "- image, row, column, channel")
print("shape of batch[0], the first image:", batch[0].shape)
print(f"numbers in the batch: {batch.size:,}")
print("the thirty labels:", labels)

fig, axes = plt.subplots(5, 6, figsize=(11, 9.8))
for k, ax in enumerate(axes.flat):
    ax.imshow(batch[k])
    ax.set_title(f"image {k + 1}, label {labels[k]}", fontsize=9)
    ax.set_xticks([])
    ax.set_yticks([])
fig.suptitle("The batch: thirty images of 96 by 96 pixels. Label 0: artwork. Label 1: space image.")
fig.tight_layout()
plt.show()
```

OUTPUT

```
shape of batch: (30, 96, 96, 3) - image, row, column, channel
shape of batch[0], the first image: (96, 96, 3)
numbers in the batch: 829,440
the thirty labels: [0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1]
```

```
grey_batch = 0.299 * batch[:, :, :, 0] + 0.587 * batch[:, :, :, 1] + 0.114 * batch[:, :, :, 2]
brightness = grey_batch.mean(axis=(1, 2))        # one number for each image
X_images = batch.reshape(30, -1)                 # one row for each image

print("shape of grey_batch:", grey_batch.shape)
print("shape of brightness:", brightness.shape)
print("shape of X_images:  ", X_images.shape)
```

```
print()
# brightness[labels == 0] selects the numbers of the images whose label is 0
print(f"mean brightness of the artworks (label 0):     {brightness[labels == 0].mean():.2f}")
print(f"mean brightness of the space images (label 1): {brightness[labels == 1].mean():.2f}")
guess = (brightness < 0.35).astype(int)          # the rule: an image darker than 0.35 is a space image
print(f"The rule 'darker than 0.35 means space image' gives the right label for {(guess == labels).sum()} of the 30
images.")
```

OUTPUT

```
shape of grey_batch: (30, 96, 96)
shape of brightness: (30,)
shape of X_images:   (30, 27648)

mean brightness of the artworks (label 0):     0.49
mean brightness of the space images (label 1): 0.30
The rule 'darker than 0.35 means space image' gives the right label for 23 of the 30 images.
```

2.7 When one operation is not enough Step 18

Every new picture in this act came from arithmetic on whole arrays, and the loop over the pixels ran inside NumPy. The last step asks for a transformation of an image that would be interesting to see and that *cannot* be written that way. Here is the kind of thing to look for: the new value of a pixel depends on new values that were already computed for other pixels, so that the pixels have to be visited one after another, in order. Then the question that the step leaves open, and that an assignment of this unit returns to: when a computation cannot be vectorized, is a slow Python loop the only choice?

Act 3. A Word Is a Vector Open the notebook of Act 3

3.1 A word as a list of numbers Step 19

A table and an image are made of numbers from the start. A word is not. Fifteen years ago it was not an accepted fact that words could be stored usefully as vectors; today every language model does it, and this act does what the first people to try it did. A **word embedding** gives every word a vector, a list of m numbers, also called its **word vector**. Nobody chose these numbers by hand. A program read a very large amount of text and computed the numbers so that words that occur in similar surroundings in the text receive similar vectors.

The vectors of this act are **GloVe** vectors, computed at Stanford University from Wikipedia and from news text, six billion words in all. The course keeps the vectors of the 50,000 most frequent English words, in two versions: with 50 numbers for each word and with 300. The version with 50 is a matrix `E50` of shape $50,000 \times 50$, of the same kind as the matrix `X` of Act 1: a row is one word and a column is one dimension. With one difference. A column of `X` had a name, a measured quantity. A column of `E50` has no name and no meaning by itself. The word of group 10 is *teacher*, in row 2,374; its fifty numbers, printed below, say nothing when they are read one at a time. The first ten rows are the ten most frequent words of English: *the*, *of*, *to*, *and*, *in*, *a*, *for*, *that*, *on*, *is*.

Code: Step 19, the matrix of word vectors, and the vector of one word cell `group-number`

```
GROUP = 10        # replace 0 with your group's number, from 1 to 30

assert 1 <= GROUP <= 30, "Type your group's number in place of the 0."

words = urlopen(EMBEDDINGS + "words.txt").read().decode().split()     # the 50,000 words, most frequent first
position = {word: k for k, word in enumerate(words)}                  # the row number of every word
E50 = read_array("glove-50.npy")

STARTERS = ["river", "guitar", "doctor", "coffee", "winter", "soccer", "piano", "bridge", "forest", "teacher",
            "ocean", "bread", "train", "garden", "mountain", "engine", "painter", "kitchen", "market", "planet",
            "island", "violin", "tiger", "hospital", "library", "airport", "chocolate", "hurricane", "bicycle",
"volcano"]
WORD = STARTERS[GROUP - 1]

print(f"E50 has shape {E50.shape}: {E50.shape[0]:,} words, and {E50.shape[1]} numbers for each word")
print("the first ten words:", words[:10])
print(f"Your group's word is '{WORD}'. It is in row {position[WORD]:,}, and its vector is:")
print(np.round(E50[position[WORD]], 2))
```

OUTPUT

```
E50 has shape (50000, 50): 50,000 words, and 50 numbers for each word
the first ten words: ['the', 'of', 'to', 'and', 'in', 'a', 'for', 'that', 'on', 'is']
Your group's word is 'teacher'. It is in row 2,374, and its vector is:
[-0.95  0.98 -0.58 -1.38  0.78  0.33 -0.44  0.2  -0.27  0.29  0.22 -0.12
  0.18 -0.2   0.28 -0.82 -0.43  1.    0.52  0.51  0.23  1.37 -0.49  0.71
  0.63 -1.71 -0.07 -1.09 -0.8   0.05  2.34 -0.29 -0.24 -0.58  0.76  0.47
  0.2   0.67  1.11  0.14 -0.18  0.39 -0.23  0.09  0.45 -0.63  0.53 -0.26
  0.02  0.9 ]
```

3.2 Which words are close to a word? Step 20

What carries information is how the vectors of two words compare. Think of each word as a point in a space with 50 dimensions; the question is which points lie near the point of *teacher*. Two vectors are compared by the **angle** between them. The **dot product** of two vectors is the sum of the products of their matching entries, $u \cdot v = \sum_k u_k v_k$. The **cosine similarity** is the dot product after each vector has been divided by its length, so that both have length 1; it is a number from -1 to 1, equal to 1 when the two vectors point in the same direction and 0 when they are perpendicular.

Two operations on whole arrays do the work. `unit_rows(E)` divides every row of the matrix by its length: `np.linalg.norm(E, axis=1, keepdims=True)` computes the 50,000 lengths, one for each row, and the division broadcasts them, as `X - mu` did in Act 1. Then `U @ v` computes the dot product of **every row** of `U` with the vector `v`: 50,000 dot products in one operation. The symbol `@` is NumPy's product of matrices and vectors, and the result holds the similarity of one word to all 50,000.

The words closest to *teacher* are *student*, with cosine similarity 0.90, then *graduate*, *teaching*, *taught*, *teaches*, *school*, *teachers* and *students*. In class the second word tried was *planet*, with the same result: every one of its neighbours was a word about planets. Nobody told the program what a teacher or a planet is. A statistical

calculation over the text put related words near each other, and it does so for any word one picks. A word has one vector whatever its number of meanings, so the neighbours of *python* are a mixture: programming languages, animals, and a group of comedians.

Code: Step 20, the nearest words cells `nearest`, `own-words`

```
def unit_rows(E):
    """Divide every row of E by its length, so that every row has length 1."""
    lengths = np.linalg.norm(E, axis=1, keepdims=True)      # one length for each row
    return E / lengths


def row(word):
    """The row number of a word."""
    assert word in position, f"'{word}' is not among the {len(words):,} words. All the words are in lower case."
    return position[word]


def nearest(word, U, k=8):
    """The k words whose vectors point most nearly in the direction of the vector of word."""
    similarity = U @ U[row(word)]                  # one dot product for every word
    order = np.argsort(-similarity)                # row numbers, from the most similar word on
    return [(words[i], round(float(similarity[i]), 2)) for i in order[1:k + 1]]    # order[0] is the word itself


U50 = unit_rows(E50)
print(f"The words closest to '{WORD}', with their cosine similarity:")
for other, similarity in nearest(WORD, U50):
    print(f"  {other:14s}{similarity:6.2f}")
```

OUTPUT

```
The words closest to 'teacher', with their cosine similarity:
  student         0.90
  graduate        0.81
  teaching        0.81
  taught          0.81
  teaches         0.79
  school          0.79
  teachers        0.78
  students        0.76
```

```
for word in ["newark", "python", "happy"]:
    print(word, "->", [other for other, similarity in nearest(word, U50)])
```

OUTPUT

```
newark -> ['albany', 'camden', 'trenton', 'minneapolis', 'wilmington', 'bridgeport', 'laguardia', 'hartford']
python -> ['php', 'owl', 'mouse', 'perl', 'monkey', 'monty', 'scripting', 'rhino']
happy -> ['everyone', 'everybody', 'really', 'me', 'definitely', 'maybe', 'feel', 'i']
```

3.3 Arithmetic on words Step 21

Vectors can be added and subtracted, and so can the vectors of words. The best-known example is

$$\text{king} - \text{man} + \text{woman} \approx \text{queen}.$$

Take the vector of *king*, subtract the vector of *man*, add the vector of *woman*. The result is a new vector, and the word whose vector is closest to it is *queen*, followed by *throne* and *prince*. Why should three vectors behave like that? Because the difference between the vectors of *king* and *man* is nearly the same vector as the difference between *queen* and *woman*: the text uses the two pairs of words in the same kinds of surroundings, and the calculation that produced the vectors preserved that. The picture in idea 7 above shows six such pairs; their arrows are nearly parallel.

The function `analogy(a, b, c, U)` answers the question *a is to b as c is to which word?*: it computes the vector of *b* minus the vector of *a* plus the vector of *c*, and returns the words closest to the result. One detail matters. The word closest to the result is often one of the three words of the question itself; without the correction, the answer to the question about *king* is *king*. The function therefore leaves the three words of the question out of its answer. *France* is to *Paris* as *Italy* is to *Rome*, then *Milan*, *Turin*; *walk* is to *walked* as *play* is to *played*; *cold* is to *colder* as *warm* is to *warmer*. Not every question is answered well. With 50 numbers for each word, *big* is to *biggest* as *small* is to *largest*, and *mouse* is to *mice* as *foot* is to *weight*. The step asks each group for one question that the vectors answer well and one that they answer badly.

Code: Step 21, analogies

cells `analogy` , `own-analogies`

```
def analogy(a, b, c, U, k=3, leave_out=True):
    """a is to b as c is to which word? The k words closest to: vector of b - vector of a + vector of c."""
    target = U[row(b)] - U[row(a)] + U[row(c)]
    similarity = U @ (target / np.linalg.norm(target))
    if leave_out:
        similarity[[row(a), row(b), row(c)]] = -2      # the three words of the question cannot be the answer
    return [words[i] for i in np.argsort(-similarity)[:k]]


print("man : king = woman : ?      ", analogy("man", "king", "woman", U50))
print("the same, nothing left out: ", analogy("man", "king", "woman", U50, leave_out=False))
print("france : paris = italy : ?  ", analogy("france", "paris", "italy", U50))
print("walk : walked = play : ?    ", analogy("walk", "walked", "play", U50))
print("cold : colder = warm : ?    ", analogy("cold", "colder", "warm", U50))
```

OUTPUT

```
man : king = woman : ?       ['queen', 'throne', 'prince']
the same, nothing left out:  ['king', 'queen', 'throne']
france : paris = italy : ?   ['rome', 'milan', 'turin']
walk : walked = play : ?     ['played', 'playing', 'players']
cold : colder = warm : ?     ['warmer', 'drier', 'cooler']
```

```
print("japan : tokyo = egypt : ?   ", analogy("japan", "tokyo", "egypt", U50))
print("big : biggest = small : ?   ", analogy("big", "biggest", "small", U50))
print("mouse : mice = foot : ?     ", analogy("mouse", "mice", "foot", U50))
```

OUTPUT

```
japan : tokyo = egypt : ?    ['cairo', 'amman', 'riyadh']
big : biggest = small : ?    ['largest', 'large', 'larger']
mouse : mice = foot : ?      ['weight', 'wound', 'stroke']
```

3.4 How many numbers for each word? Step 22

The number m of numbers that each word receives is the **dimension** of the embedding, chosen before the vectors are computed. With more numbers for each word, the vectors can record more distinctions between words. The price is the one that Act 1 measured: a matrix with n rows and m columns holds nm numbers, and every operation on it visits all of them. The matrix with 50 numbers for each word occupies 10 megabytes and the one with 300 occupies 60; comparing one word with all 50,000 took 0.22 milliseconds with 50 numbers and 0.84 with 300.

The notebook puts the same forty analogy questions to both versions, of six kinds: a country and its capital, a man and a woman, the forms of an adjective, the forms of a verb, one and many, a country with its language or its money. The version with 50 numbers answered 29 of the 40 correctly and the version with 300 answered 35. The larger version corrects *mouse* : *mice* = *foot* : *feet* and *man* : *woman* = *uncle* : *aunt*, and it still gets some wrong: *small* is to *largest* in both, and with 300 numbers *Canada* is to *Ottawa* as *Peru* is to *Peruvian*. More numbers for each word buy better answers and cost memory and time; they do not make every answer right. The neighbours of a word also sharpen: with 300 numbers, *teacher* gains *schoolteacher*, *classroom* and *instructor*.

Code: Step 22, fifty against three hundred numbers for each word cells `dimension`, `own-300`

```
E300 = read_array("glove-300.npy")
U300 = unit_rows(E300)
print(f"E50  has shape {E50.shape}: {E50.size:,} numbers, {E50.nbytes / 1e6:.0f} megabytes")
print(f"E300 has shape {E300.shape}: {E300.size:,} numbers, {E300.nbytes / 1e6:.0f} megabytes")

# Each line of four words reads: the first is to the second as the third is to the fourth.
QUESTIONS = [
    ("france", "paris", "italy", "rome"), ("france", "paris", "japan", "tokyo"), ("germany", "berlin", "spain",
"madrid"),
    ("russia", "moscow", "egypt", "cairo"), ("greece", "athens", "poland", "warsaw"), ("china", "beijing", "ireland",
"dublin"),
    ("england", "london", "austria", "vienna"), ("sweden", "stockholm", "norway", "oslo"), ("turkey", "ankara", "cuba",
"havana"),
    ("canada", "ottawa", "peru", "lima"),
    ("man", "woman", "king", "queen"), ("man", "woman", "brother", "sister"), ("man", "woman", "uncle", "aunt"),
    ("man", "woman", "father", "mother"), ("man", "woman", "son", "daughter"), ("man", "woman", "actor", "actress"),
    ("boy", "girl", "prince", "princess"), ("he", "she", "his", "her"),
    ("big", "bigger", "small", "smaller"), ("good", "better", "bad", "worse"), ("fast", "faster", "slow", "slower"),
    ("strong", "stronger", "weak", "weaker"), ("big", "biggest", "small", "smallest"), ("cold", "colder", "warm",
"warmer"),
    ("long", "longer", "short", "shorter"), ("high", "highest", "low", "lowest"),
    ("walk", "walked", "play", "played"), ("go", "went", "take", "took"), ("see", "saw", "give", "gave"),
    ("run", "running", "swim", "swimming"), ("eat", "ate", "write", "wrote"), ("sing", "sang", "speak", "spoke"),
    ("car", "cars", "dog", "dogs"), ("child", "children", "man", "men"), ("city", "cities", "country", "countries"),
    ("mouse", "mice", "foot", "feet"),
    ("france", "french", "spain", "spanish"), ("japan", "japanese", "germany", "german"), ("italy", "italian", "greece",
"greek"),
    ("japan", "yen", "russia", "ruble"),
]

answers50 = [analogy(a, b, c, U50, k=1)[0] for a, b, c, d in QUESTIONS]
answers300 = [analogy(a, b, c, U300, k=1)[0] for a, b, c, d in QUESTIONS]
correct50 = sum(answer == d for answer, (a, b, c, d) in zip(answers50, QUESTIONS))
correct300 = sum(answer == d for answer, (a, b, c, d) in zip(answers300, QUESTIONS))
```

```
print()
print(f"Correct answers out of {len(QUESTIONS)}: with 50 numbers for each word {correct50}, with 300 numbers
{correct300}")
print()
print("The questions on which the two versions give different answers:")
print(f"  {'question':38s}{'expected':12s}{'50 numbers':14s}{'300 numbers':14s}")
for (a, b, c, d), small, large in zip(QUESTIONS, answers50, answers300):
    if small != large:
        print(f"  {a + ' : ' + b + ' = ' + c + ' : ?':38s}{d:12s}{small:14s}{large:14s}")

print()
for name, U in (("50 numbers", U50), ("300 numbers", U300)):
    v = U[row(WORD)]
    start = time.perf_counter()
    for _ in range(20):
        U @ v
    seconds = (time.perf_counter() - start) / 20
    print(f"{name:12s}: comparing one word with all 50,000 words takes {seconds * 1000:.2f} milliseconds")
```

OUTPUT

```
E50  has shape (50000, 50): 2,500,000 numbers, 10 megabytes
E300 has shape (50000, 300): 15,000,000 numbers, 60 megabytes

Correct answers out of 40: with 50 numbers for each word 29, with 300 numbers 35

The questions on which the two versions give different answers:
  question                               expected    50 numbers    300 numbers
  germany : berlin = spain : ?           madrid      rome          madrid
  england : london = austria : ?         vienna      stockholm     vienna
  canada : ottawa = peru : ?             lima        lima          peruvian
  man : woman = uncle : ?                aunt        daughter      aunt
  long : longer = short : ?              shorter     usually       shorter
  eat : ate = write : ?                  wrote       writing       wrote
  child : children = man : ?             men         who           men
  mouse : mice = foot : ?                feet        weight        feet

50 numbers  : comparing one word with all 50,000 words takes 0.22 milliseconds
300 numbers : comparing one word with all 50,000 words takes 0.84 milliseconds
```

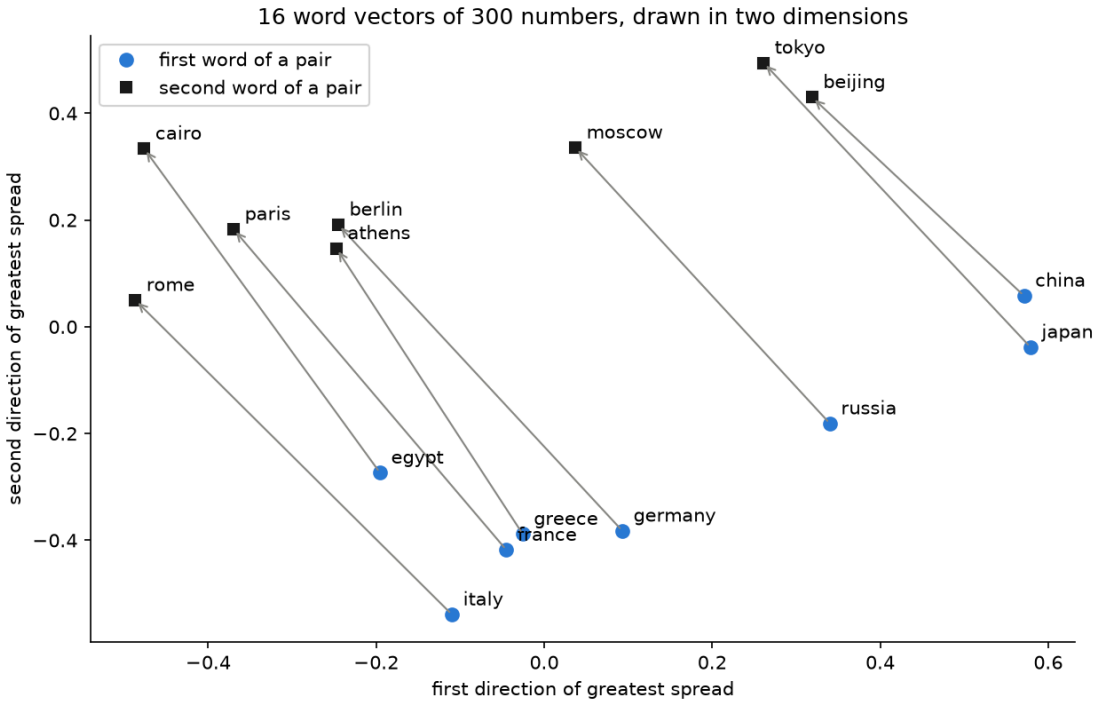
```
print(f"closest to '{WORD}', 50 numbers: ", [other for other, similarity in nearest(WORD, U50)])
print(f"closest to '{WORD}', 300 numbers:", [other for other, similarity in nearest(WORD, U300)])
print("big : biggest = small : ?   ", analogy("big", "biggest", "small", U300))
print("mouse : mice = foot : ?     ", analogy("mouse", "mice", "foot", U300))
```

OUTPUT

```
closest to 'teacher', 50 numbers:  ['student', 'graduate', 'teaching', 'taught', 'teaches', 'school',
'teachers', 'students']
closest to 'teacher', 300 numbers: ['teachers', 'student', 'teaching', 'taught', 'school', 'schoolteacher',
'classroom', 'instructor']
big : biggest = small : ?    ['largest', 'tiny', 'large']
mouse : mice = foot : ?      ['feet', 'legs', 'ankle']
```

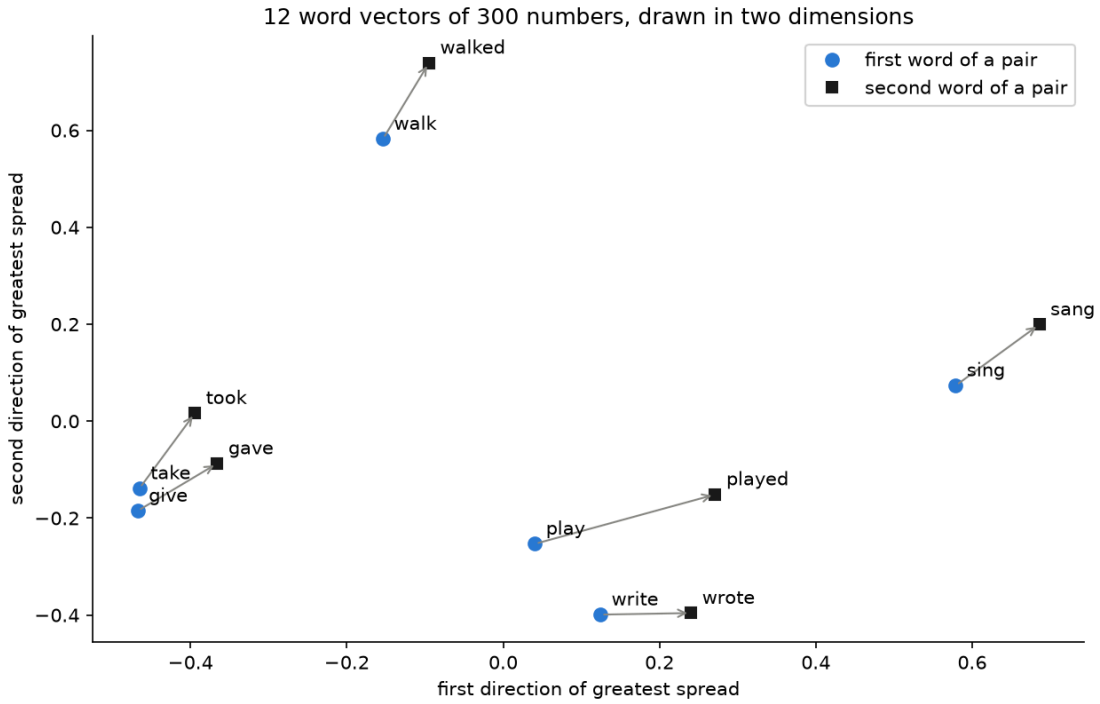
3.5 A picture of word vectors Step 23

A vector of 300 numbers cannot be drawn. For a small set of words a picture is still possible: find the **two directions** along which the chosen vectors are most spread out, and draw each word at its two coordinates along those directions. `np.linalg.svd` finds the directions. The picture keeps two of the 300 dimensions, so distances in it are approximate, a shadow of the real arrangement. The picture below draws eight countries and their capitals, with an arrow from each country to its capital. If the difference between a country and its capital were the same vector for every pair, the arrows would be parallel. They are nearly so, and the notebook says so in numbers before it draws: for each pair, the cosine similarity between its arrow and the average of the other seven arrows, computed with all 300 numbers, lies between 0.66 and 0.78.



Eight countries and their capitals, drawn in the two directions of greatest spread.

The step asks each group for a picture of its own: five to eight pairs that share one relation. Below is the notebook's own example, six verbs and their past forms. The arrows are less parallel than those of the capitals; their similarities lie between 0.50 and 0.70. The relation between a verb and its past tense is real in the vectors, but it is spread over more directions than the relation between a country and its capital.



Six verbs and their past forms.

Code: Step 23, the picture

cells `picture`, `own-picture`

```
def picture(pairs, U):
    """Draw the vectors of the words of the pairs in two dimensions, with an arrow from the first word of a pair to the
second."""
    chosen = [word for pair in pairs for word in pair]
    V = U[[row(word) for word in chosen]]
    V = V - V.mean(axis=0)
    directions = np.linalg.svd(V, full_matrices=False)[2][:2]      # the two directions of greatest spread
    xy = V @ directions.T                                          # two coordinates for each word
    place = dict(zip(chosen, xy))

    arrows = np.array([U[row(b)] - U[row(a)] for a, b in pairs])
    print("Cosine similarity of each arrow with the average of the other arrows (1 means parallel):")
    for k, (a, b) in enumerate(pairs):
        others = np.delete(arrows, k, axis=0).mean(axis=0)
        cosine = arrows[k] @ others / np.linalg.norm(arrows[k]) / np.linalg.norm(others)
        print(f"  {a:12s}-> {b:12s}{cosine:6.2f}")

    fig, ax = plt.subplots(figsize=(8.5, 5.5))
    for a, b in pairs:
        ax.annotate("", xy=place[b], xytext=place[a], arrowprops=dict(arrowstyle="->", color="#8a8a86"))
        ax.annotate(a, place[a], textcoords="offset points", xytext=(6, 5))
        ax.annotate(b, place[b], textcoords="offset points", xytext=(6, 5))
    firsts = np.array([place[a] for a, b in pairs])
    seconds = np.array([place[b] for a, b in pairs])
    ax.plot(firsts[:, 0], firsts[:, 1], "o", color="#2a78d6", markersize=7, label="first word of a pair")
    ax.plot(seconds[:, 0], seconds[:, 1], "s", color="#1a1a19", markersize=6, label="second word of a pair")
    ax.set_title(f"{len(chosen)} word vectors of {U.shape[1]} numbers, drawn in two dimensions")
    ax.set_xlabel("first direction of greatest spread")
    ax.set_ylabel("second direction of greatest spread")
    ax.spines[["top", "right"]].set_visible(False)
```

```
    ax.legend()
    fig.tight_layout()
    plt.show()


PAIRS = [("france", "paris"), ("italy", "rome"), ("japan", "tokyo"), ("germany", "berlin"),
         ("russia", "moscow"), ("egypt", "cairo"), ("greece", "athens"), ("china", "beijing")]
picture(PAIRS, U300)
```

OUTPUT

```
Cosine similarity of each arrow with the average of the other arrows (1 means parallel):
  france      -> paris         0.76
  italy       -> rome          0.68
  japan       -> tokyo         0.73
  germany     -> berlin        0.72
  russia      -> moscow        0.78
  egypt       -> cairo         0.71
  greece      -> athens        0.68
  china       -> beijing       0.66
```

```
OUR_PAIRS = [("walk", "walked"), ("play", "played"), ("give", "gave"), ("take", "took"), ("write", "wrote"), ("sing",
"sang")]
picture(OUR_PAIRS, U300)
```

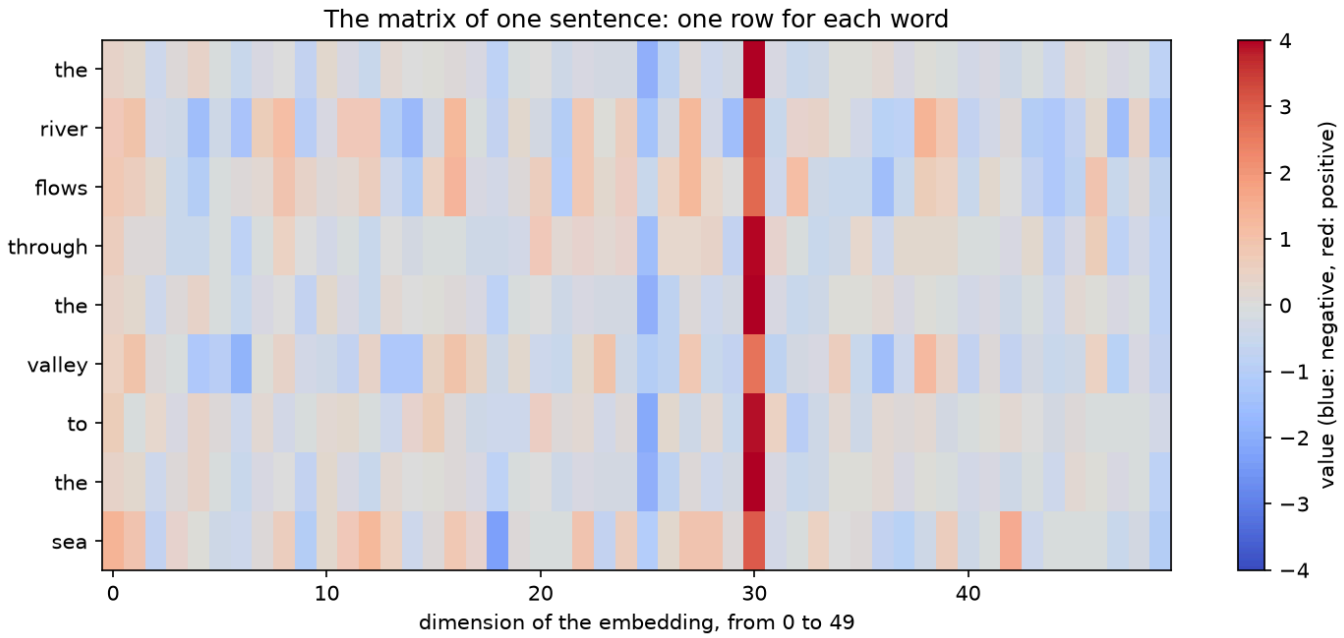
OUTPUT

```
Cosine similarity of each arrow with the average of the other arrows (1 means parallel):
  walk        -> walked        0.53
  play        -> played        0.52
  give        -> gave          0.70
  take        -> took          0.70
  write       -> wrote         0.50
  sing        -> sang          0.52
```

3.6 A text is a sequence of vectors Step 24

A text is words in an order. When each word is replaced by its vector, the text becomes a **sequence of vectors**: a matrix with one row for each word, in the order of the text. A text of L words gives a matrix of shape $L \times m$. In the table of Act 1 the order of the rows carried no information; here it does. A text is also one point of a dataset, as one image was in Act 2, and it can carry a label, such as its language or its subject.

The sentence *the river flows through the valley to the sea* has nine words and becomes a matrix of shape 9×50. Drawn as a picture, one row for each word and one column for each of the fifty dimensions, with a colour for each value, the matrix is the same kind of object as the grey image of Act 2. A word has the same row wherever it occurs: the two rows of *the* are alike. The sentences *the dog chased the cat* and *the cat chased the dog* have the same words and therefore the same rows, in a different order, so their matrices differ; but the averages of their rows are equal. An operation that ignores the order of the rows, such as their average, cannot tell the two sentences apart. That is why the order has to be kept.



The matrix of one sentence: one row for each word, fifty columns.

Code: Step 24, a sentence as a matrix

cell `sentence`

```
def text_matrix(text, E):
    """The matrix of a text: one row for each word, in the order of the text."""
    return E[[row(word) for word in text.lower().split()]]


SENTENCE = "the river flows through the valley to the sea"       # your turn: a sentence of your own, without punctuation
S = text_matrix(SENTENCE, E50)
print(f"'{SENTENCE}'")
print(f"shape of S: {S.shape}, which is {S.shape[0]} words, and {S.shape[1]} numbers for each word")

largest = float(np.abs(S).max())
fig, ax = plt.subplots(figsize=(10, 0.36 * len(S) + 1.3))
drawing = ax.imshow(S, cmap="coolwarm", vmin=-largest, vmax=largest, aspect="auto")
ax.set_yticks(range(len(S)))
ax.set_yticklabels(SENTENCE.lower().split())
ax.set_xlabel("dimension of the embedding, from 0 to 49")
ax.set_title("The matrix of one sentence: one row for each word")
fig.colorbar(drawing, ax=ax, label="value (blue: negative, red: positive)")
fig.tight_layout()
plt.show()

first = text_matrix("the dog chased the cat", E50)
second = text_matrix("the cat chased the dog", E50)
print("'the dog chased the cat' and 'the cat chased the dog':")
print("   rows 0 and 3 of the first matrix, both the word 'the', are equal:", np.array_equal(first[0], first[3]))
print("   the two matrices are equal:", np.array_equal(first, second))
print("   the averages of their rows are equal:", np.allclose(first.mean(axis=0), second.mean(axis=0)))
```

OUTPUT

```
'the river flows through the valley to the sea'
shape of S: (9, 50), which is 9 words, and 50 numbers for each word
'the dog chased the cat' and 'the cat chased the dog':
   rows 0 and 3 of the first matrix, both the word 'the', are equal: True
   the two matrices are equal: False
   the averages of their rows are equal: True
```

3.7 A batch of texts Step 25

Several texts are stored together as a batch, like the images of Act 2, and one difficulty is new: texts have different lengths, and an array needs the same number of rows for every text. The remedy is **padding**. Every text shorter than the longest one is filled up, at its end, with rows of zeros. A batch of B texts, padded to L words, with m numbers for each word, is an array with three dimensions, $B \times L \times m$: text, position in the text, dimension of the embedding. The three sentences of the notebook, of nine, three and eight words, become an array of shape $3 \times 9 \times 50$, with six rows of zeros after the short sentence and one after the last. Programs that process language, large language models among them, receive text in exactly this form: a batch of sequences of vectors.

Code: Step 25, a padded batch of texts cell `text-batch`

```python
def text_batch(texts, E):
    """Stack the matrices of several texts into one array, padded with rows of zeros to the longest text."""
    matrices = [text_matrix(text, E) for text in texts]
    longest = max(len(M) for M in matrices)
    batch = np.zeros((len(texts), longest, E.shape[1]), dtype=E.dtype)
    for k, M in enumerate(matrices):
        batch[k, :len(M)] = M                    # the rows after the last word stay zero
    return batch


TEXTS = ["the river flows through the valley to the sea",          # your turn: three sentences of your own
         "students write code",
         "she scored two goals in the final match"]
T = text_batch(TEXTS, E50)
print("shape of T:", T.shape, "- text, position in the text, dimension of the embedding")
for k, text in enumerate(TEXTS):
    count = len(text.split())
    print(f"   text {k}: {count} words and {T.shape[1] - count} rows of zeros: '{text}'")
print(f"numbers in the batch: {T.size:,}")
```

OUTPUT

```
shape of T: (3, 9, 50) - text, position in the text, dimension of the embedding
   text 0: 9 words and 0 rows of zeros: 'the river flows through the valley to the sea'
   text 1: 3 words and 6 rows of zeros: 'students write code'
   text 2: 8 words and 1 rows of zeros: 'she scored two goals in the final match'
numbers in the batch: 1,350
```

What the vectors take from their text.

The vectors record which words occur near which in Wikipedia and in news text, and nothing else. They therefore reproduce the associations of that text, including its stereotypes: a question about professions and genders, for example, is answered as the text answers it, not as it should be. This is a property of the data, and every system built on such vectors inherits it. The course's 50,000 words leave out a list of slurs.